Project Demonstration

This document is a visual, step-by-step walkthrough of the complete machine learning pipeline — from data generation to prediction explanation — including real terminal output captured from running each script. It serves as a written substitute for a live demo.

Workflow Overview

```
Generate Dataset
        ↓
Train & Evaluate Model
        ↓
Explain Predictions
```

- **Generate Dataset** — simulates synthetic shipment data with a documented delay formula.
- **Train & Evaluate Model** — fits a Random Forest classifier and validates it with cross-validation.
- **Explain Predictions** — computes global and shipment-level explanations from the trained model.

Each stage below corresponds to one script in `src/`, run in order from the project root.

Step 1 — Generate Dataset

```
python src/generate_data.py
```

What it does: simulates 500 inbound shipments — vessel type, weather, queue length, port utilization, and historical route delay — and derives each shipment's outcome from a documented, seeded formula (see `docs/dataset_design.md`).

Output location: `src/data/port_data.csv`

Expected output: a summary of row count, delay rate, and feature distributions, so the dataset can be sanity-checked immediately after generation.

src/generate_data.py

```
$ python src/generate_data.py
============================================================
Synthetic Dataset Summary
============================================================
Rows: 500
Delayed Rate: 37.00%

Weather Distribution
weather
Clear    0.682
Rain     0.212
Storm    0.106
Name: proportion, dtype: float64

Vessel Distribution
vessel_type
Container    0.498
Bulk         0.316
Tanker       0.186
Name: proportion, dtype: float64

First Five Rows
  vessel_type weather  queue_length  ...  arrival_hour  delay_hours  is_delayed
0        Bulk    Rain            22  ...            16        28.27           1
1   Container    Rain             1  ...            12         8.68           0
2      Tanker   Storm            10  ...            23        34.65           1
3        Bulk   Clear            11  ...            18        25.68           1
4   Container   Clear            20  ...             0        17.45           0

[5 rows x 8 columns]
```

Figure 1: generate_data.py output

Step 2 — Train the Model

```
python src/train_model.py
```

What it does: one-hot encodes categorical features, removes leakage columns (`delay_hours`, `is_delayed`), and evaluates a Random Forest classifier with Stratified 5-Fold Cross Validation — preserving the delay/no-delay ratio in every fold. The final model is refit on the full dataset and saved.

Output location: `models/model.joblib`, `models/feature_schema.json`, `models/metrics.json`

Expected output: per-fold ROC-AUC/F1, mean and out-of-fold metrics, and the out-of-fold confusion matrix.

src/train_model.py

```
$ python src/train_model.py
============================================================
Model Training Summary (5-Fold CV)
============================================================
Fold 1: ROC-AUC=0.949  F1=0.806
Fold 2: ROC-AUC=0.973  F1=0.836
Fold 3: ROC-AUC=0.984  F1=0.907
Fold 4: ROC-AUC=0.947  F1=0.849
Fold 5: ROC-AUC=0.944  F1=0.818

Mean ROC-AUC: 0.959
Mean F1:      0.843

Out-of-fold metrics:
ROC-AUC: 0.958
F1:      0.844

Confusion matrix @ threshold=0.5:
  TN=296  FP=19
  FN=36  TP=149
```

Figure 2: train_model.py output

Step 3 — Explain the Model

```
python src/explain_model.py
```

What it does: loads the trained model (no retraining), ranks features by two independent importance methods, and generates plain-language explanations for individual shipments.

Output location: `models/feature_importance.json`

Global feature importance — both the Random Forest's built-in importance and permutation importance are printed side by side, so the two can be compared directly:

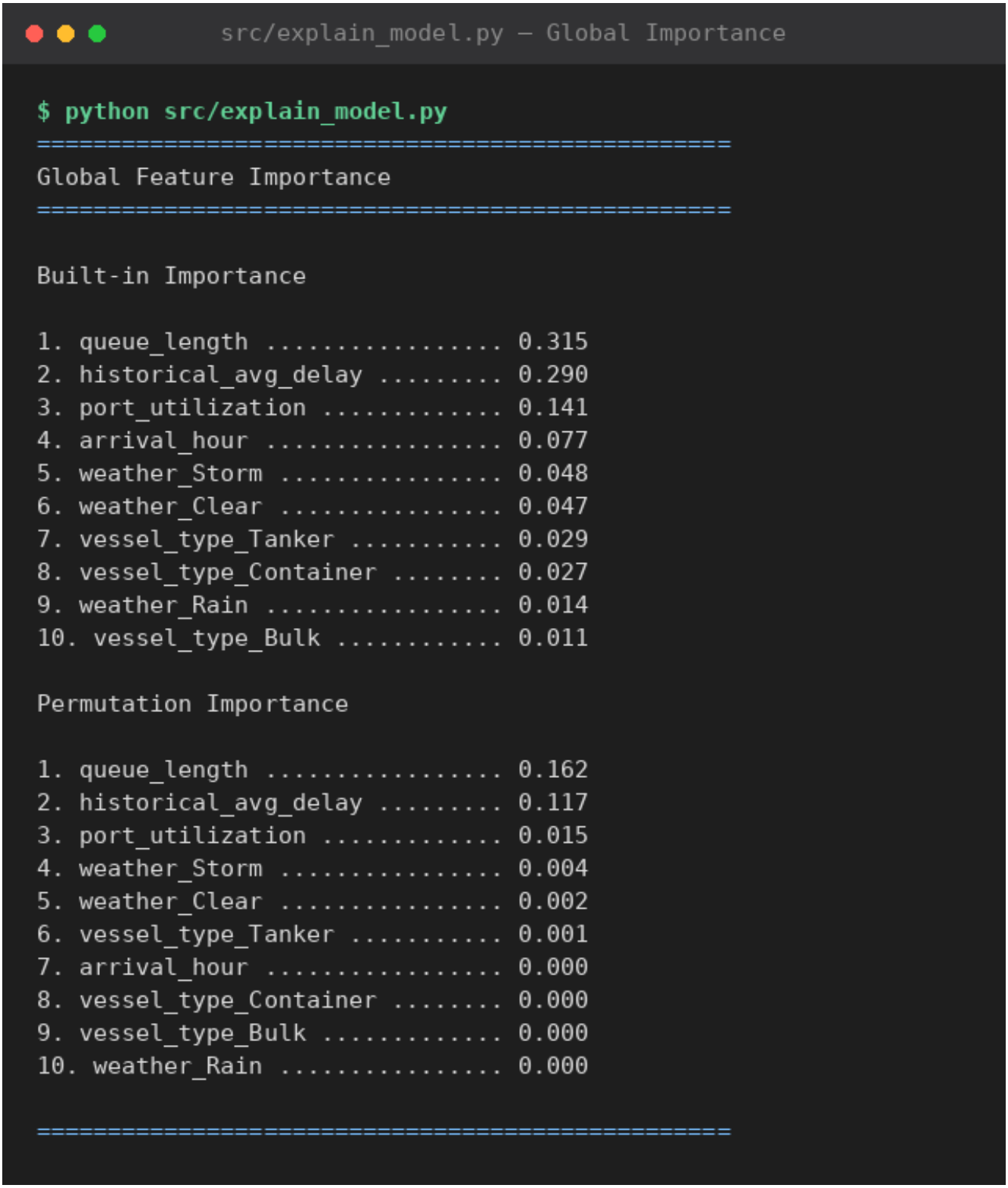

Figure 3: explain_model.py global importance output

Shipment-level explanations and sanity check — two worked examples (highest and lowest predicted risk), followed by an automated check confirming that `arrival_hour` — a feature documented as carrying

no real signal — is correctly identified as low-importance by permutation importance, despite ranking artificially higher under built-in importance:

src/explain_model.py — Shipment Explanations

```
$ python src/explain_model.py
================================================
Example Shipment Explanations
================================================

Shipment #105

Prediction

High Delay Risk (100%)

Explanation

* Queue length is significantly higher than normal, indicating heavy congestion at the port.
  This is the single biggest factor in this prediction.
* This route has a recent history of longer-than-average delays.
* The port is operating close to full capacity, making delays more likely.

Overall, this shipment is assessed as high delay risk (100% predicted probability of delay).

Shipment #2

Prediction

Low Delay Risk (0%)

Explanation

* Queue length is lower than normal, indicating a smoothly flowing port. This is the single
  biggest factor in this prediction.
* This route has a strong recent on-time track record.
* The port has considerable spare capacity right now, reducing the likelihood of delay.

Overall, this shipment is assessed as low delay risk (0% predicted probability of delay).


================================================
Sanity Check: arrival_hour
================================================
Built-in importance:     rank 4/10  (value=0.077)
Permutation importance:  rank 7/10  (value=0.000)

Built-in feature importance ranks 'arrival_hour' higher than expected, while permutation
  importance shows almost no predictive contribution. This discrepancy is expected because
  impurity-based feature importance in Random Forest can overestimate continuous numerical
  features.
```

Figure 4: explain_model.py shipment explanations output

Generated Artifacts

Artifact	Description
`model.joblib`	Trained Random Forest model
`metrics.json`	Cross-validation metrics
`feature_schema.json`	Feature ordering for inference
`feature_importance.json`	Explainability results

End-to-End Pipeline Summary

1. Generate a synthetic logistics dataset.
2. Train and evaluate a Random Forest classifier.
3. Save the trained model.
4. Compute feature importance.
5. Explain individual shipment predictions.

Key Takeaways

- End-to-end, reproducible ML pipeline — every stage is seeded and deterministic.
- Strong predictive performance (0.959 mean ROC-AUC) on the synthetic dataset it was built and validated on.
- Explainable predictions suitable for operational decision support.
- Lightweight implementation using only the Python scientific ecosystem.

Notes

No graphical user interface was developed — this project focuses on the machine learning workflow rather than frontend presentation. The screenshots above are real, unedited console output captured by running each script in sequence, and together with the command-line pipeline and accompanying documentation, serve as the project demonstration.